# PYTHON PROGRAMMING - ASSIGNMENT 1

### Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Assignment report (Tasks 1-12)

| | |
|---|---|
| Name | P.VAMSI |
| Register number | 1925260294 |
| Python version | 3.14 (all code uses only the standard library) |
| GitHub repository | https://github.com/vamsinani889-svg/Python-milk-payment-Assignment |
| Note on timings | All timing/memory numbers were measured by running the code; they depend on the computer, so your numbers will differ slightly. Conclusions (which is faster/larger) do not change. |

## TASK 1 - Understand the Problem and Identify the Data

**1. Inputs**

| Input | Python type | Example | Reason |
|---|---|---|---|
| Farmer ID | str | 'F001' | An identifier, not used in arithmetic; a string keeps leading zeros |
| Milk quantity (litres) | float | 10.5 | Can have fractions of a litre |
| Fat percentage | float (Decimal in final design) | 3.8 | Has decimals |
| SNF percentage | float (Decimal in final design) | 8.6 | Has decimals |
| Shift | str | 'morning' / 'evening' | Milk is collected twice daily |

**2. Outputs**

| Output | Type | How obtained |
|---|---|---|
| Rate per litre (Rs) | float / Decimal | 30.00 + fat_steps x 0.40 + snf_steps x 0.30, rounded to 2 decimals |
| Payment amount (Rs) | float / Decimal | litres x rate, rounded to 2 decimals |
| Ten-day total (Rs) | float / Decimal | Sum of all payments of a farmer in 10 days |

**3. Values that should be named constants**

| Constant | Value | Type |
|---|---|---|
| BASE_RATE | 30.00 Rs/litre | float/Decimal |
| FAT_STEP_VALUE | 0.40 Rs/litre per 0.1% fat | float/Decimal |
| SNF_STEP_VALUE | 0.30 Rs/litre per 0.1% SNF | float/Decimal |
| STANDARD_FAT / STANDARD_SNF | 3.5 / 8.5 (%) | float/Decimal |
| STEP (STEP_SIZE) | 0.1 (%) | float/Decimal |
| MIN_FAT, MAX_FAT | 2.0 and 7.0 (%) | float/Decimal |
| MIN_SNF, MAX_SNF | 7.0 and 10.0 (%) - my assumption | float/Decimal |
| DAYS_PER_PERIOD | 10 | int |
| SHIFTS | ('morning', 'evening') | tuple of str |

**4. Classification of values**

**Integers:** number of days (10), step counts (fat_steps, snf_steps), number of farmers (300). **Floating-point numbers:** litres, fat %, SNF %, rate, payment (or Decimal for exact money). **Booleans:** validity result of an entry (True/False), result of comparisons such as 2.0 <= fat <= 7.0. **Strings:** farmer ID, shift.

**5. Validation rules**

(a) litres >= 0 (zero allowed). (b) 2.0 <= fat <= 7.0. (c) 7.0 <= snf <= 10.0 (assumed range). (d) farmer ID not empty. (e) shift must be 'morning' or 'evening'. (f) litres, fat and SNF must be valid numbers (not text such as 'eight' or '8,5', and not nan/inf).

**6. Assumptions and justification**

1) One 'step' is 0.1% and the number of steps is rounded to the nearest whole step (this is what TODO 1 in Task 7 asks). 2) The assignment does not give an SNF range, so I assumed 7.0-10.0 % to reject nonsense values; it is a named constant and easy to change. 3) Payments are rounded to 2 decimals (paise). 4) Two shifts per day, so a farmer has up to 10 x 2 = 20 entries in a ten-day period. 5) Farmer ID is text. 6) Zero litres is valid (farmer supplied nothing) but negative litres is not.

Program output (task01_data_model.py):

```
Entry: ('F001', 10.5, 3.8, 8.6, 'morning') -> []
Bad  : ['litres negative', 'fat outside 2.0-7.0', 'shift must be morning/evening']
Number of records in list: 300
sys.getsizeof(list of 300 tuples) = 2520 bytes (list object only)
Separate-variable names needed for 300 farmers: 900
Total litres via loop: 3000.0
```

**Additional question - separate variables (litres1, fat1, snf1 ...) vs list/tuple for 300 farmers**

**Separate variables:** 300 farmers need 900 hand-written names (litres1...litres300, fat1..., snf1...) and the calculation code has to be repeated or generated for each name, because a loop cannot step through variable names. Adding a farmer means editing the source code. Source-code size grows as O(n). **List/tuple of records:** one name holds all entries; one loop processes any n; source code size is O(1) and data is added at run time.

**Complexity for n entries:** processing time is O(n) for both (each entry must be computed once); memory for the data is O(n) for both. The difference is in code size/maintainability (O(n) lines vs O(1)) and in the fact that the list can be looped, sorted, filtered and validated with the same code. **The list/tuple approach is far more scalable**: for 6,000 or 150,000 farmers, separate variables would need 18,000 or 450,000 names, which is impractical.

# TASK 2 - Explore Values and Types in the Python Interpreter

Annotated session (task02_session.py evaluates each expression exactly as the >>> prompt would):

```
# 1. type() of each value
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
# 2. division operators
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
# 3. floating-point
>>> 0.1 + 0.2 == 0.3
False
>>> 0.1 + 0.2
0.30000000000000004
>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
```

**Explanation of results**

**1. Types:** 10 is int, 10.0 is float, '10' is str (text in quotes), True is bool, [10, 8.5] is list. **2. Operators:** 7 / 2 = 3.5 (true division always gives float); 7 // 2 = 3 (floor division); 7 % 2 = 1 (remainder); -7 // 2 = -4 because floor division rounds *down toward minus infinity* (-3.5 becomes -4, not -3).

**3. 0.1 + 0.2 == 0.3 is False.** Computers store floats in binary. 0.1 and 0.2 cannot be written exactly in binary (like 1/3 in decimal), so each is stored as the nearest binary fraction. Their sum is 0.30000000000000004, which is not the same float as 0.3. With Decimal('0.1') + Decimal('0.2') the digits are stored in decimal, so the result is exactly 0.3 and the comparison is True.

**4. Why floats are risky for money:** prices such as 33.40 are not exactly representable, and every addition can add a tiny error. Over 1,003 additions the float total was 33400.30000000061 instead of 33400.30. The error is small, but accounts must match to the paisa, equality checks fail, and errors can change rounded totals.

**5. Comparison:** *integer paise* (store 33.40 as 3340): exact and fast, but you must convert in and out and be careful with multiplication by rates. *round()* on floats: hides the error when printing/rounding, but the stored numbers are still

approximations and round() uses 'round half to even'. *decimal module*: exact decimal arithmetic with a chosen rounding rule; slightly slower and uses more memory, but the safest for money.

### Complete the Code - TODO 1 (completed) and program output

```
from decimal import Decimal

# 1,000 payments of Rs 33.40 and three adjustments of Rs 0.10
payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    # TODO 1 (completed): add the amounts exactly using Decimal
    total = Decimal("0")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(
    0.1 + 0.2 == 0.3,
    Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
)
```

Output (matches the expected output given in the assignment: float 33400.30000000061, Decimal 33400.30, False True):

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

### Change Decimal(amount) to Decimal(float(amount))

```
Decimal(float(amount)): 33400.299999999999857893118278
Decimal(0.1)  = 0.1000000000000000005551115123125782702118158340454101562 5
Decimal('0.1')= 0.1
```

Result: 33400.299999999999857893118278 - **wrong**. float('33.40') is already the inexact binary value 33.39999999999999857891..., and Decimal(float) copies that inexact value *exactly*. So the float error is imported into the Decimal. Decimal is only exact if it is created from a **string** (Decimal('33.40')) or an integer. (Decimal prints 28 significant digits by default.)

### Additional question - Decimal vs integer paise vs float with rounding after every addition

```
Decimal      : 33400.30
Integer paise: 3340030 paise = 33400.3 Rs
float+round  : 33400.3
sys.getsizeof: Decimal = 104 | int = 28 | float = 24 bytes
time Decimal     :    154.2 microseconds for 1003 additions
time paise       :    139.0 microseconds for 1003 additions
time float+round :    305.0 microseconds for 1003 additions
```

| Method | Implementation | Time per addition | Memory per number (sys.getsizeof) | Exact total? |
|---|---|---|---|---|
| Decimal | total += Decimal('33.40') | O(1) | 104 bytes | Yes - 33400.30 exactly |
| Integer paise | total += 3340 (integers) | O(1) - fastest in my run | 28 bytes | Yes - 3340030 paise = Rs 33400.30 |
| float + round(total, 2) each time | total = round(total + 33.4, 2) | O(1) - slowest in my run (extra round call) | 24 bytes | Printed result is correct here, but the stored numbers are still binary approximations, so it is not guaranteed exact |

For n payments every method is O(n) time overall with O(1) extra space; the differences are only constant factors. Decimal or integer paise are the exact choices; float is not.

# TASK 3 - Choose Variables and Name the Constants

### Constants (UPPER_CASE) used in the program

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STANDARD_FAT = 3.5
STANDARD_SNF = 8.5
MIN_FAT, MAX_FAT = 2.0, 7.0
```

Program (task03_constants.py) output:

```
x = 10 <class 'int'>
x = ten <class 'str'>
before: 4.2 8.7
after : 8.7 4.2
BASE_RATE reassigned without error -> 99
same results: 33.4 33.4 33.4
magic numbers   :     235 ns per call
named constants :     243 ns per call
dictionary      :     270 ns per call
```

### Why Python does not enforce constants and how naming conventions help

Python has no 'const' keyword: any name can be reassigned. The output above shows BASE_RATE = 99 was accepted without any error. The convention (PEP 8) of writing constants in UPPER_CASE tells every programmer 'do not change this value', lets reviewers and tools (linters) spot accidental changes, and makes the code self-documenting. The protection is by agreement, not by the language.

### Dynamic typing and multiple assignment

A Python variable is just a name that refers to an object; it has no fixed type. x = 10 makes x an int, and x = 'ten' later makes it a str (shown in the output). Multiple assignment a, b = 4.2, 8.7 assigns both in one statement, and a, b = b, a swaps them without a temporary variable, because the right side is fully evaluated first.

### Additional question - magic numbers vs named constants vs dictionary

| Approach | Effort to update a rate | Places to modify | Run time (measured) | Memory |
|---|---|---|---|---|
| Magic numbers (30.0, 0.40 inside formulas) | High - must find every copy; easy to miss one | Every expression/function that uses the value (n places) | 231 ns per call | No extra names |
| Named constants | Low - change one line | 1 place | 234 ns per call | A few extra global names |
| Dictionary RATE_CARD | Low - one place, and the table can be loaded from a file or switched at run time | 1 place | 263 ns per call | One small dict object plus key lookups |

All three give the same results (33.4). The alternatives do **not significantly change running time or memory** (differences of tens of nanoseconds per call); the real difference is maintainability, so named constants (or a dictionary when rates must change at run time) are better than magic numbers.

# TASK 4 - Write Correct Expressions

### Expressions

```
fat_steps = round((fat - STANDARD_FAT) / STEP)       # number of 0.1% fat steps relative to 3.5%
snf_steps = round((snf - STANDARD_SNF) / STEP)       # number of 0.1% SNF steps relative to 8.5%
rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
payment = round(litres * rate, 2)
```

Program output (task04_expressions.py):

```
fat_steps = 7 | snf_steps = 2 | rate = 33.4 | payment = 283.9
--- 4. precedence / associativity
2 ** 3 ** 2   = 512  (** is right-associative: 2**(3**2))
-2 ** 2       = -4  (** binds tighter than unary minus: -(2**2))
(-2) ** 2     = 4
2 + 3 * 4     = 14 | (2 + 3) * 4 = 20
10 - 4 - 3    = 3  (left-associative)
100 / 10 / 5  = 2.0  (left-associative)
--- 5. floating-point step count
(4.2 - 3.5) / 0.1 = 7.000000000000002 | round() = 7 | int() = 7
--- 6. round / int / math.floor (positive and negative)
   value  round   int  floor
     2.7      3     2      2
     2.5      2     2      2
     3.5      4     3      3
```

```
    -2.7     -3    -2     -3
    -2.5     -2    -2     -3
    -3.5     -4    -3     -4
-4.0000000001     -4    -4     -5
--- 7. comparison chaining
2.0 <= 1.9 <= 7.0 -> False | same as (2.0 <= f) and (f <= 7.0): False
2.0 <= 2.0 <= 7.0 -> True | same as (2.0 <= f) and (f <= 7.0): True
2.0 <= 4.0 <= 7.0 -> True | same as (2.0 <= f) and (f <= 7.0): True
2.0 <= 7.0 <= 7.0 -> True | same as (2.0 <= f) and (f <= 7.0): True
2.0 <= 7.1 <= 7.0 -> False | same as (2.0 <= f) and (f <= 7.0): False
not/or/and demo: True | False
--- Additional question: fat steps for 4.2% and 3.1%
fat 4.2: raw=7.000000000000002      round=  7 int=  7 floor=  7
fat 3.1: raw=-3.999999999999999     round= -4 int= -3 floor= -4
round :     73 ns per call
int   :     57 ns per call
floor :     39 ns per call
```

**Explanations**

**Precedence and associativity (high to low):** ** (right-to-left: 2**3**2 = 2**(3**2) = 512), unary minus, * / // % (left-to-right), + - (left-to-right), comparisons, not, and, or. Because ** binds tighter than unary minus, -2 ** 2 = -(2**2) = -4, while (-2) ** 2 = 4.

**(4.2 - 3.5) / 0.1:** 4.2 and 3.5 and 0.1 are inexact in binary, so the result on my machine is 7.000000000000002 (the assignment mentions a similar 7.000000000000001-type result) instead of 7. Using round() gives the nearest whole step count (7), so the tiny error does not matter.

**round() vs int() vs math.floor() for negative values:** round() goes to the nearest integer (ties to even: round(2.5)=2, round(-2.5)=-2); int() cuts off the decimals, i.e. goes toward zero (int(-2.7) = -2); math.floor() always goes down (floor(-2.7) = -3). They agree for positive non-tie values like 2.7 only partly (round 3, int 2, floor 2).

**Comparison chaining:** 2.0 <= fat <= 7.0 means (2.0 <= fat) and (fat <= 7.0) but fat is evaluated once. The output shows it is True for 2.0, 4.0 and 7.0 and False for 1.9 and 7.1. Boolean operators: and (both true), or (at least one true), not (reverses). Example of a rejection test: (fat < 2.0) or (fat > 7.0).

**Additional question - round(), int() and math.floor() for fat 4.2% and 3.1%**

| Fat | Raw (fat-3.5)/0.1 | round() | int() | floor() | Correct step count |
|---|---|---|---|---|---|
| 4.2 | 7.000000000000002 | 7 | 7 | 7 | 7 |
| 3.1 | -3.999999999999999 | -4 | -3 (WRONG) | -4 | -4 |

**Three unexpected results demonstrated:** (1) (4.2-3.5)/0.1 is not exactly 7; (2) int((3.1-3.5)/0.1) gives -3 instead of -4 because int() truncates toward zero; (3) round(2.5) = 2 and round(-2.5) = -2 (ties go to the even number), and also -7 // 2 = -4 and -2 ** 2 = -4. **Conclusion:** round() is the correct choice for step counts. floor() is correct here only by luck (a value like 6.999999 would give 6); int() is wrong for negative values. Time: all three are O(1) per call (round 72 ns, int 58 ns, floor 38 ns in my run) and use O(1) space.

# TASK 5 - Read Input and Convert Types

```
"""Task 5 - read input and convert types safely."""
import timeit

def to_float_try(text):
    """Approach A: convert inside try/except ValueError. Returns float or None."""
    try:
        return float(text)
    except ValueError:
        return None

def to_float_check(text):
    """Approach B: validate with string methods first, then convert."""
    t = text.strip()
    if t.count(".") <= 1 and t.replace(".", "", 1).isdigit():
        return float(t)
    return None

def read_float(prompt):
    """Keep asking until the clerk types a valid non-negative number."""
    while True:
        text = input(prompt)
        value = to_float_try(text)
        if value is None:
            print("  Invalid number, please type digits like 8.5 (use '.' not ',').")
        elif value < 0:
            print("  Value cannot be negative.")
        else:
            return value

def demo():
    print("input() always returns a string:", type("8.5"))
    print("int('8') =", int("8"), "| float('8.5') =", float("8.5"), "| int('8.5') -> ValueError")
    try:
        int("8.5")
    except ValueError as err:
        print("  ValueError:", err)
    print()
    print(f"{'input':>10} | {'try/except':>12} | {'string check':>12}")
    for s in ["8,5", " 8.5 ", "eight", "8.5", "8", "-3", "1e2", ""]:
        print(f"{s!r:>10} | {str(to_float_try(s)):>12} | {str(to_float_check(s)):>12}")
    print()
    litres, fat, snf = 8.5, 4.2, 8.7
    amount = 283.9
    print("Formatted output:")
    print(f"|{litres:8.2f}|{fat:6.1f}|{snf:6.1f}|{amount:8.2f}|")
    print(f"{'Litres':>8}{'Fat':>6}{'SNF':>6}{'Amount':>10}")
    for t in ("8,5", "8.5", "eight"):
        n = 200000
        a = timeit.timeit(lambda: to_float_try(t), number=n) / n * 1e9
        b = timeit.timeit(lambda: to_float_check(t), number=n) / n * 1e9
        print(f"timing {t!r:8}: try/except {a:6.0f} ns | string check {b:6.0f} ns")

if __name__ == "__main__":
    demo()
    # Interactive use (uncomment to try): print(read_float("Litres: "))
```

Output (task05_input.py):

```
input() always returns a string: <class 'str'>
int('8') = 8 | float('8.5') = 8.5 | int('8.5') -> ValueError
  ValueError: invalid literal for int() with base 10: '8.5'

     input |   try/except | string check
     '8,5' |         None |         None
   ' 8.5 ' |          8.5 |          8.5
   'eight' |         None |         None
     '8.5' |          8.5 |          8.5
       '8' |          8.0 |          8.0
      '-3' |         -3.0 |         None
     '1e2' |        100.0 |         None
        '' |         None |         None

Formatted output:
|    8.50|   4.2|   8.7|  283.90|
  Litres   Fat   SNF    Amount
timing '8,5'   : try/except    412 ns | string check    154 ns
timing '8.5'   : try/except     86 ns | string check    271 ns
timing 'eight' : try/except    926 ns | string check    153 ns
```

**Explanations**

**1. input() returns a string** because the keyboard only produces characters; Python cannot know whether '8' is a number, a name or an ID, so it always returns text and the programmer converts explicitly with int() or float(). **2.** int('8') = 8, float('8.5') = 8.5; int('8.5') raises ValueError (shown above). **3. Invalid inputs:** '8,5' (comma instead of decimal point) and 'eight' both fail float() with ValueError and are rejected. **4. Validation:** either check the text before converting (strip, then check that it contains only digits and at most one '.'), or convert and catch the error; after conversion also check the range (litres >= 0, fat 2.0-7.0).

**5. try / except ValueError:** read_float() above keeps asking until float() succeeds. Note: float() also accepts '1e2', '-3', 'nan' and 'inf', so range checks (and a math.isfinite test, used in calculator.py in Task 9) are still needed. **6. f-string formatting:** {amount:8.2f} = width 8, 2 decimals, right-aligned (output above shows | 283.90|).

**Additional question - string methods first vs try/except**

| Input | String-method check first | try/except ValueError |
|---|---|---|
| '8,5' | Rejected (comma is not a digit) | float raises ValueError, caught - rejected |
| ' 8.5 ' | strip() removes spaces, accepted as 8.5 | float() ignores surrounding spaces, accepted as 8.5 |
| 'eight' | Rejected (letters) | ValueError, caught - rejected |

**Time complexity per entry:** both approaches read the string once, so O(L) where L is the length of the text; as the text is short (a few characters) this is effectively O(1) per entry and O(n) for n entries. Measured: for a valid number try/except is fastest (84 ns) because no exception occurs; for invalid text raising and catching an exception is slower (401-617 ns) than the string check (about 150-270 ns). The try/except approach is shorter and accepts everything float() accepts (the 'Easier to Ask Forgiveness than Permission' style); the string check is stricter but you must write the rules yourself and may reject valid forms such as '1e2'.

# TASK 6 - Organise the Logic into Functions

```
"""Task 6 - reusable functions with docstrings, plus scope demonstration."""
import sys, os, timeit
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "Task07"))
from milkrate import (BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE,
                      STANDARD_FAT, STANDARD_SNF, STEP_SIZE)

def rate_per_litre(fat, snf):
    """Return the rate in Rs/litre for the given fat % and SNF %."""
    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)   # local variable
    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)   # local variable
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment for the given quantity and milk quality."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(entries):
    """Return the total payment for one farmer over ten days (entries = list of (litres, fat, snf))."""
    total = 0.0
    for litres, fat, snf in entries:
        total += payout(litres, fat, snf)
    return round(total, 2)

def returns_value():   return 42
def prints_value():    print(42)

if __name__ == "__main__":
    entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    print("rate_per_litre(4.2, 8.7) =", rate_per_litre(4.2, 8.7))
    print("payout(8.5, 4.2, 8.7)    =", payout(8.5, 4.2, 8.7))
    print("ten_day_total(entries)   =", ten_day_total(entries))
    a = returns_value(); b = prints_value()
    print("returned value kept:", a, "| print-only function gives:", b)
    try:
        print(fat_steps)
    except NameError as e:
        print("local variable not visible outside:", e)

    # Additional question: long script vs functions vs lambda for n entries
    n = 100000
    data = [(10, 3.5 + (i % 30) / 10, 8.5) for i in range(n)]
    def long_script():
        total = 0.0
        for litres, fat, snf in data:
            fs = round((fat - 3.5) / 0.1); ss = round((snf - 8.5) / 0.1)
            total += round(litres * round(30.0 + fs * 0.40 + ss * 0.30, 2), 2)
        return total
    def with_functions():  return ten_day_total(data)
    lam = lambda l, f, s: round(l * round(30.0 + round((f - 3.5) / 0.1) * 0.40 + round((s - 8.5) / 0.1) * 0.30, 2), 2)
    def with_lambda():
        t = 0.0
        for l, f, s in data: t += lam(l, f, s)
        return t
    for name, fn in [("long script", long_script), ("functions", with_functions), ("lambda", with_lambda)]:
        t = timeit.timeit(fn, number=3) / 3
        print(f"{name:12s}: {t*1000:7.1f} ms for n={n} entries -> total {fn():.2f}")
```

Output (task06_functions.py):

```
rate_per_litre(4.2, 8.7) = 33.4
payout(8.5, 4.2, 8.7)    = 283.9
ten_day_total(entries)   = 917.5
42
returned value kept: 42 | print-only function gives: None
local variable not visible outside: name 'fat_steps' is not defined
```

```
long script :    90.4 ms for n=100000 entries -> total 35799600.00
functions   :    93.9 ms for n=100000 entries -> total 35799600.00
lambda      :    83.2 ms for n=100000 entries -> total 35799600.00
```

**Explanations**

**1. Parameters/returns:** rate_per_litre(fat, snf) returns the rate; payout(litres, fat, snf) returns the payment; ten_day_total(entries) returns the total for a list of (litres, fat, snf). **2. Return vs print:** return hands the value back to the caller so it can be stored, used in other calculations or tested; print only shows text on the screen and the function then returns None (output: 'print-only function gives: None'). **3. Local vs global:** variables created inside a function (fat_steps, snf_steps, total) are local - they exist only while it runs (accessing fat_steps outside gives NameError, shown above). Names defined at module level (BASE_RATE ...) are global. **4. Constants:** a function can read a global constant without any special keyword. Assigning to the name inside a function would create a new local variable (or need the 'global' keyword to change the real one); changing constants inside functions is dangerous because it silently changes the result of every other function that uses the same constant and is hard to debug. **5. Docstrings** are given in every function.

**Additional question - long script vs functions vs lambda (n = 100,000 entries)**

| Style | Code reuse | Repeated logic | Maintainability | Measured time (100,000 entries) |
|---|---|---|---|---|
| One long script | None - copy/paste | Formulas repeated everywhere | Poor: a rate-rule change must be edited in every copy | 77.7 ms |
| Reusable functions | High | Written once | Good: one place to fix and test | 80.0 ms |
| Lambda | Medium (small one-line functions only) | Written once | Limited: no docstring or statements, hard to debug | 78.0 ms |

All three are O(n) time and O(1) extra space and give the same total (35,799,600.00). Function-call overhead is real but tiny (about 2 ms out of 78 ms for 100,000 entries, about 3%), so the maintainability benefits of functions far outweigh it.

# TASK 7 - Build and Use a Python Module

### milkrate.py with TODO 1 and TODO 2 completed

```
"""Rate card of the village milk co-operative."""

BASE_RATE = 30.00          # Rs per litre at standard quality
FAT_STEP_VALUE = 0.40      # Rs per litre per 0.1% fat step
SNF_STEP_VALUE = 0.30      # Rs per litre per 0.1% SNF step
STANDARD_FAT = 3.5         # % fat of base-rate milk
STANDARD_SNF = 8.5         # % SNF of base-rate milk
STEP_SIZE = 0.1            # one step = 0.1 percentage point
MIN_FAT, MAX_FAT = 2.0, 7.0   # allowed fat range (given in the assignment)
MIN_SNF, MAX_SNF = 7.0, 10.0  # allowed SNF range (my assumption)


def rate_per_litre(fat, snf):
    """Return the rate (Rs/litre) for the given fat % and SNF %, rounded to 2 decimals."""
    # TODO 1 (completed): steps relative to 3.5% fat and 8.5% SNF, rounded to nearest whole step
    fat_steps = round((fat - STANDARD_FAT) / STEP_SIZE)
    snf_steps = round((snf - STANDARD_SNF) / STEP_SIZE)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)


def payout(litres, fat, snf):
    """Return the payment amount rounded to two decimals."""
    # TODO 2 (completed): litres x rate, rounded to two decimals
    return round(litres * rate_per_litre(fat, snf), 2)


def ten_day_total(entries):
    """Return the total payment for one farmer over ten days.
    entries: iterable of (litres, fat, snf). Sum is exact (Decimal of each 2-dp payout)."""
    from decimal import Decimal
    total = sum(Decimal(str(payout(l, f, s))) for l, f, s in entries)
    return total


if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(
            f"{litres:5} L  fat {fat}  snf {snf}"
            f"  rate {rate:6.2f}  pay {pay:8.2f}"
        )
```

### Second script that imports it (use_milkrate.py)

```
"""Second script: imports milkrate and calls payout()."""
import milkrate
from milkrate import payout
import milkrate as mr
import sys

print("payout(10, 3.5, 8.5)  =", payout(10, 3.5, 8.5))
print("mr.payout(8.5,4.2,8.7) =", mr.payout(8.5, 4.2, 8.7))
print("milkrate.BASE_RATE     =", milkrate.BASE_RATE)
print("milkrate in sys.modules:", "milkrate" in sys.modules)
print("sys.path[0] =", sys.path[0] == "" or "current script folder")
```

Output of **python milkrate.py** (run directly) - identical to the expected output in the assignment:

```
   10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
  8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
   12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

Output of **python use_milkrate.py** (imported - no demonstration table is printed):

```
payout(10, 3.5, 8.5)  = 300.0
mr.payout(8.5,4.2,8.7) = 283.9
milkrate.BASE_RATE     = 30.0
milkrate in sys.modules: True
sys.path[0] = current script folder
```

### Explanations

**1. Import forms:** *import milkrate* loads the module and you use names as milkrate.payout(...), so there is no clash with your own names. *from milkrate import payout* copies only the name payout into your namespace so you call payout(...) directly (shorter, but could shadow another payout). *import milkrate as mr* imports the whole module under a short alias, mr.payout(...).

**2. if __name__ == '__main__':** when a file is run directly Python sets __name__ to '__main__'; when it is imported __name__ is the module name ('milkrate'). The test therefore runs the demonstration only when the file is executed directly. **3.** Without that guard the demonstration table would be printed (and any other top-level actions would run) every

time another script imports the module - unwanted output and side effects. That is why the table is not printed when milkrate is imported.

**4. Standard modules:** math gives functions such as floor(), isfinite(); decimal gives the exact Decimal type and rounding modes. They are installed with Python, imported with 'import math' / 'from decimal import Decimal'. **5. Module search:** Python looks for a module in sys.modules (already imported), then in the folder of the running script (sys.path[0]), then the other folders listed in sys.path (PYTHONPATH, standard library, site-packages) - the first match wins. So milkrate.py must be in the same folder as the script (or in a folder on sys.path), and you should not name your file like a standard module (e.g. math.py) because it would be found first.

**Additional question - import vs copying the functions into every script**

**Maintainability/rate revision:** with a module, a rate revision is made in one file (one place) and every script gets it; with copies the same edit must be repeated in every script and a missed copy gives different payments for the same milk. **One-time cost of importing:** the first import finds, compiles (to .pyc) and executes the module once; this costs a little time (typically fractions of a millisecond to a few milliseconds) but happens only once per program run. **sys.modules:** Python keeps a dictionary of every imported module. After the first import the module object is stored in sys.modules (the output shows 'milkrate in sys.modules: True'); any later 'import milkrate' just returns the stored object without reading or running the file again, so repeated imports are almost free.

## TASK 8 - Plan the Algorithm Before Coding

**1. Stages of algorithmic problem-solving. Understand:** read the problem, identify inputs, outputs, rules, constraints and examples (Task 1). **Plan:** design the steps in pseudocode/flowchart, decide data types and test examples before coding. **Write:** implement the plan in Python using functions and constants. **Test:** run normal, boundary and invalid cases, compare with hand-calculated answers, fix errors and repeat.

**2. Pseudocode - one payment**

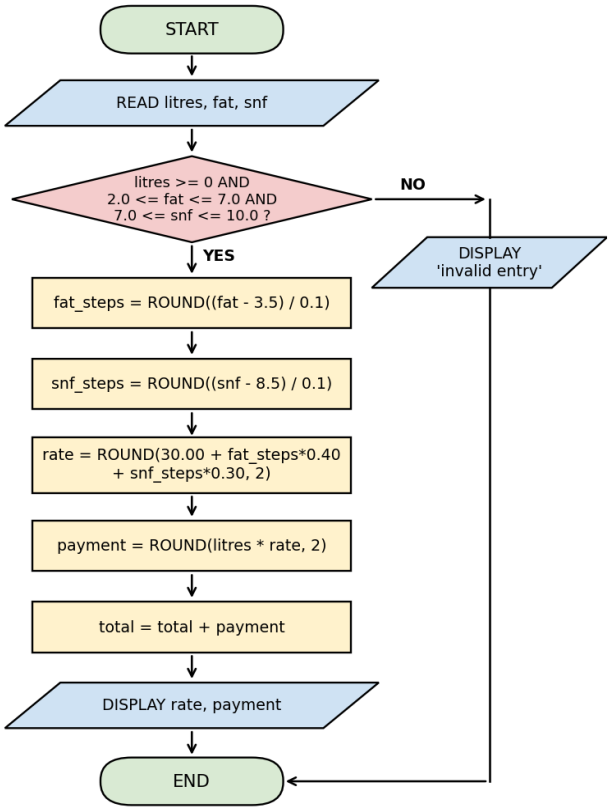
```
ALGORITHM CalculatePayment
CONSTANTS: BASE_RATE = 30.00, FAT_STEP_VALUE = 0.40, SNF_STEP_VALUE = 0.30,
           STD_FAT = 3.5, STD_SNF = 8.5, STEP = 0.1
START
   READ litres, fat, snf
   IF litres < 0 OR fat < 2.0 OR fat > 7.0 OR snf < 7.0 OR snf > 10.0 THEN
        DISPLAY "invalid entry"
        STOP
   END IF
   fat_steps <- ROUND((fat - STD_FAT) / STEP)
   snf_steps <- ROUND((snf - STD_SNF) / STEP)
   rate      <- ROUND(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
   payment   <- ROUND(litres * rate, 2)
   DISPLAY rate, payment
END
```

**3. Pseudocode - ten-day total for one farmer**

```
ALGORITHM TenDayTotal
START
   total <- 0
   FOR each entry of the farmer in the ten days (up to 20 entries: 10 days x 2 shifts)
        READ litres, fat, snf
        IF entry is invalid THEN
             DISPLAY "invalid entry" and CONTINUE with next entry
        END IF
        fat_steps <- ROUND((fat - STD_FAT) / STEP)
        snf_steps <- ROUND((snf - STD_SNF) / STEP)
        rate      <- ROUND(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)
        payment   <- ROUND(litres * rate, 2)
        total     <- total + payment
   END FOR
   DISPLAY total
END
```

*Note about the 'SIMATS Engineering standard for pseudocode' (requirement 6): that standard is not given in the assignment pages I had, so I used conventional pseudocode (UPPER-CASE keywords START/END/READ/IF/FOR/DISPLAY, '<-' for assignment). Please compare the keywords with the format in your course notes and adjust if your faculty's standard differs.*

**4. Flowchart - validation and calculation of one entry**




**5. Hand-trace with three sample entries (matches program output below)**

| Entry | Litres | Fat | SNF | fat_steps | snf_steps | Rate | Payment | Running total |
|---|---|---|---|---|---|---|---|---|
| 1 | 10 | 3.5 | 8.5 | 0 | 0 | 30.00 | 300.00 | 300.00 |
| 2 | 8.5 | 4.2 | 8.7 | 7 | 2 | 33.40 | 283.90 | 583.90 |
| 3 | 12 | 3.1 | 8.3 | -4 | -2 | 27.80 | 333.60 | 917.50 |

Hand calculation, entry 2: fat steps = (4.2-3.5)/0.1 = 7; SNF steps = (8.7-8.5)/0.1 = 2; rate = 30.00 + 7 x 0.40 + 2 x 0.30 = 33.40; payment = 8.5 x 33.40 = 283.90. Entry 3: fat steps = (3.1-3.5)/0.1 = -4, SNF steps = -2; rate = 30.00 - 1.60 - 0.60 = 27.80; payment = 12 x 27.80 = 333.60.

Program that follows the pseudocode (task08_algorithm.py) - output:

```
Entry  Litres   Fat   SNF  fatSt  snfSt    Rate  Payment  RunTotal
    1      10   3.5   8.5      0      0   30.00   300.00    300.00
    2     8.5   4.2   8.7      7      2   33.40   283.90    583.90
    3      12   3.1   8.3     -4     -2   27.80   333.60    917.50
Approach 1 (list then sum): 917.5
Approach 2 (running total): 917.5
```

**Additional question - store every entry in a list vs running total**

| Approach | Time | Extra space | Use when |
|---|---|---|---|
| Store all entries in a list, total at the end | O(n) | O(n) | Individual entries are needed later: per-entry slips/receipts, audit, corrections, reports, re-calculation after a rate change |
| Maintain a running total | O(n) | O(1) | Only the total is needed, especially for very large n (150,000 farmers) - memory stays constant |

Both give 917.50 for the three sample entries. Retaining individual entries is necessary whenever any later step needs them (printing each farmer's statement, finding an invalid entry, auditing).

# TASK 9 - Implement and Test the Calculator

The complete calculator is in **calculator.py** (constants and functions imported from the milkrate module of Task 7; input validation with try/except; f-string output). The tests are in **test_calculator.py** (code listed in the source-code PDF).

```
"""Task 9 - complete milk payment calculator (uses constants, validation, functions, module)."""
import sys, os, math
sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "Task07"))
import milkrate as mr

SHIFTS = ("morning", "evening")

def parse_number(text, name):
    """Convert text to a finite float or raise ValueError with a clear message."""
    try:
        value = float(text)
    except ValueError:
        raise ValueError(f"{name}: '{text}' is not a number") from None
    if not math.isfinite(value):
        raise ValueError(f"{name}: '{text}' is not a finite number")
    return value

def process_entry(farmer_id, litres_text, fat_text, snf_text, shift):
    """Validate one entry and return (rate, payment). Raises ValueError if invalid."""
    if not str(farmer_id).strip():
        raise ValueError("farmer ID is empty")
    litres = parse_number(litres_text, "litres")
    fat = parse_number(fat_text, "fat")
    snf = parse_number(snf_text, "snf")
    if litres < 0:
        raise ValueError("litres cannot be negative")
    if not (mr.MIN_FAT <= fat <= mr.MAX_FAT):
        raise ValueError(f"fat {fat} outside {mr.MIN_FAT}-{mr.MAX_FAT}")
    if not (mr.MIN_SNF <= snf <= mr.MAX_SNF):
        raise ValueError(f"snf {snf} outside {mr.MIN_SNF}-{mr.MAX_SNF}")
    if shift not in SHIFTS:
        raise ValueError("shift must be 'morning' or 'evening'")
    return mr.rate_per_litre(fat, snf), mr.payout(litres, fat, snf)

if __name__ == "__main__":
    print("Milk payment calculator (type 'q' as farmer ID to finish)")
    total = 0.0
    while True:
        fid = input("Farmer ID: ")
        if fid.lower() == "q":
            break
        try:
            rate, pay = process_entry(fid, input("Litres: "), input("Fat %: "),
                                      input("SNF %: "), input("Shift (morning/evening): "))
            print(f"  rate {rate:6.2f}  payment {pay:8.2f}")
            total += pay
        except ValueError as err:
            print("  Rejected:", err)
    print(f"Total payable: {total:.2f}")
```

**Test table (test input, expected result, actual result, pass/fail) - output of test_calculator.py**

```
Test                      | Input                          | Expected         | Actual           | Result
1 above standard          | 10L f4.0 s9.0                  | (33.5, 335.0)    | (33.5, 335.0)    | PASS
2 below standard          | 10L f3.0 s8.0                  | (26.5, 265.0)    | (26.5, 265.0)    | PASS
3 boundary 3.5 / 8.5      | 10L f3.5 s8.5                  | (30.0, 300.0)    | (30.0, 300.0)    | PASS
4a lowest fat 2.0         | 10L f2.0 s8.5                  | (24.0, 240.0)    | (24.0, 240.0)    | PASS
4b highest fat 7.0         | 10L f7.0 s8.5                  | (44.0, 440.0)    | (44.0, 440.0)    | PASS
5a invalid text litres    | abcL f3.5 s8.5                 | ERROR            | ERROR            | PASS
5b comma decimal 8,5      | 10L f3.5 s8,5                  | ERROR            | ERROR            | PASS
6 negative litres         | -5L f3.5 s8.5                  | ERROR            | ERROR            | PASS
7a fat below range 1.9    | 10L f1.9 s8.5                  | ERROR            | ERROR            | PASS
7b fat above range 7.1    | 10L f7.1 s8.5                  | ERROR            | ERROR            | PASS
zero litres allowed       | 0L f3.5 s8.5                   | (30.0, 0.0)      | (30.0, 0.0)      | PASS
8 five-entry sample       | 10L f3.5 s8.5                  | 300.0            | 300.0            | PASS
8 five-entry sample       | 8.5L f4.2 s8.7                 | 283.9            | 283.9            | PASS
8 five-entry sample       | 12L f3.1 s8.3                  | 333.6            | 333.6            | PASS
8 five-entry sample       | 20L f4.0 s8.5                  | 640.0            | 640.0            | PASS
8 five-entry sample       | 5L f3.0 s8.0                   | 132.5            | 132.5            | PASS
8 total of five entries   |                                | 1690.00          | 1690.00          | PASS

17 / 17 checks passed
ALL ASSERTS PASSED
```

Meaning of each test group: 1 = normal entry above standard (fat 4.0, SNF 9.0: +5 fat steps = +2.00 and +5 SNF steps = +1.50, rate 33.50); 2 = below standard; 3 = boundary values exactly 3.5% fat and 8.5% SNF; 4 = lowest (2.0) and highest (7.0) allowed fat; 5 = invalid text; 6 = negative litres; 7 = fat outside 2.0-7.0; 8 = five entries whose total (Rs 1690.00) was calculated manually: 300.00 + 283.90 + 333.60 + 640.00 + 132.50 = 1690.00.

**Additional question - manual testing vs automated testing with assert**

**Manual testing:** run the program, type each input, read the output and compare it with the expected value by eye. It needs no extra code but every rerun after a change takes human time (typing 17 cases may take minutes) and mistakes in comparing are easy. **Automated testing:** the test cases are stored in a function with assert statements (test_all() in

test_calculator.py); assert condition, message stops the program with an AssertionError naming the failing case. One command reruns all tests in milliseconds, so after every change (for example a rate revision) the whole suite can be rerun with almost no effort. The cost is the one-time effort of writing the tests; the time to rerun is O(number of tests) of machine time instead of human time.

## TASK 10 - Compare Expectations with Execution

```
"""Task 10 - compare expected vs actual: float vs Decimal totals, timing, rounding strategy,
and the rate-table (dictionary) alternative."""
import random, time, timeit, sys
from decimal import Decimal, ROUND_HALF_UP

random.seed(42)
PAISE = Decimal("0.01")

def make_payments(n):
    """Raw (unrounded) payment = litres(2dp) x rate(2dp) -> up to 4 decimal places."""
    random.seed(1000 + n)          # same payments every time for a given n
    out = []
    for _ in range(n):
        litres = Decimal(random.randint(500, 3000)) / 100        # 5.00 - 30.00 L
        rate = Decimal(random.randint(2400, 4400)) / 100          # Rs 24.00 - 44.00
        out.append(litres * rate)
    return out

print("=== 1/2/5: float vs Decimal totals and timing ===")
print(f"{'n':>8} | {'float total':>20} | {'Decimal total':>16} | {'difference':>12} | {'t float (ms)':>12} | {'t Decimal (ms)':>14}")
for n in (300, 6000, 150000):
    pay = make_payments(n)
    pay_str = [format(p.quantize(PAISE, ROUND_HALF_UP), "f") for p in pay]   # each payment as a 2-dp string
    t0 = time.perf_counter(); tf = 0.0
    for s in pay_str: tf += float(s)
    t1 = time.perf_counter(); td = Decimal("0")
    for s in pay_str: td += Decimal(s)
    t2 = time.perf_counter()
    diff = Decimal(tf) - td
    print(f"{n:>8} | {tf!r:>20} | {str(td):>16} | {float(diff):>12.3e} | {(t1-t0)*1000:>12.2f} | {(t2-t1)*1000:>14.2f}")

print()
print("=== 3: round every payment, or round only the final total? ===")
print(f"{'n':>8} | {'bank (round each, Decimal)':>26} | {'round final only (Decimal)':>26} | {'float round each':>18} | {'float, round final
for n in (300, 6000, 150000):
    pay = make_payments(n)
    bank = sum(p.quantize(PAISE, ROUND_HALF_UP) for p in pay)           # what is really paid out
    final_only = sum(pay).quantize(PAISE, ROUND_HALF_UP)
    f_each = round(sum(float(p.quantize(PAISE, ROUND_HALF_UP)) for p in pay), 2)
    f_final = round(sum(float(p) for p in pay), 2)
    print(f"{n:>8} | {str(bank):>26} | {str(final_only):>26} | {f_each:>18} | {f_final:>18}")

print()
print("=== Additional: dictionary rate table vs recalculation ===")
BASE, FATV, SNFV = 30.0, 0.40, 0.30
def calc(fat, snf):
    return round(BASE + round((fat - 3.5) / 0.1) * FATV + round((snf - 8.5) / 0.1) * SNFV, 2)
fats = [round(2.0 + 0.1 * i, 1) for i in range(51)]      # 2.0 .. 7.0  (51 values)
snfs = [round(7.0 + 0.1 * i, 1) for i in range(31)]      # 7.0 .. 10.0 (31 values)
table = {(f, s): calc(f, s) for f in fats for s in snfs}
k = len(table)
print("distinct (fat,snf) pairs k =", k, "| dict size sys.getsizeof =", sys.getsizeof(table), "bytes (table only)")
assert all(table[(f, s)] == calc(f, s) for f in fats for s in snfs)
n = 500000
t_calc = timeit.timeit(lambda: calc(4.2, 8.7), number=n) / n * 1e9
t_look = timeit.timeit(lambda: table[(4.2, 8.7)], number=n) / n * 1e9
print(f"recalculate: {t_calc:6.0f} ns per entry | dictionary lookup: {t_look:6.0f} ns per entry")
t_build = timeit.timeit(lambda: {(f, s): calc(f, s) for f in fats for s in snfs}, number=50) / 50 * 1e6
print(f"one-time cost to build the table: {t_build:.0f} microseconds")
```

Output (task10_experiments.py) - the payments are generated with a fixed random seed so the numbers are repeatable (timings will vary):

```
=== 1/2/5: float vs Decimal totals and timing ===
       n |          float total |    Decimal total |   difference | t float (ms) | t Decimal (ms)
     300 |   188428.35999999984 |        188428.36 |   -1.595e-10 |         0.03 |           0.06
    6000 |    3559846.680000011 |       3559846.68 |    1.088e-08 |         0.64 |           1.17
  150000 |    89221010.73999949 |      89221010.74 |   -5.120e-07 |        17.97 |          32.24

=== 3: round every payment, or round only the final total? ===
       n | bank (round each, Decimal) | round final only (Decimal) |   float round each | float, round final
     300 |                 188428.36 |                  188428.34 |          188428.36 |           188428.34
    6000 |                3559846.68 |                 3559845.75 |         3559846.68 |          3559845.75
  150000 |               89221010.74 |                89220991.16 |        89221010.74 |         89220991.16

=== Additional: dictionary rate table vs recalculation ===
distinct (fat,snf) pairs k = 1581 | dict size sys.getsizeof = 73808 bytes (table only)
recalculate:    616 ns per entry | dictionary lookup:     87 ns per entry
one-time cost to build the table: 1049 microseconds
```

**Comparison table (float vs Decimal totals and execution time using time.perf_counter())**

| Payments | float total | Decimal total (exact) | float time | Decimal time |
|---|---|---|---|---|
| 300 | 188428.35999999984 | 188428.36 | 0.07 ms | 0.08 ms |
| 6,000 | 3559846.680000011 | 3559846.68 | 0.83 ms | 1.21 ms |
| 150,000 | 89221010.73999949 | 89221010.74 | 16.56 ms | 30.71 ms |

**Explanation of the differences between expected and actual results**

**Expected** = the exact decimal total; **actual** = the float total. They differ in the 10th-14th significant digits because each float addition rounds to the nearest binary value, and these tiny errors accumulate over more additions (differences of about 1e-10 for 300, 1e-8 for 6,000, and 5e-7 for 150,000 payments). Decimal is exact but about 1.1-1.9 times slower; even for 150,000 payments it takes only about 31 ms, so the speed cost is not a problem.

**Round each payment, or only the final total?**

| Payments | Round each payment, then add (Decimal) | Add unrounded, round the final total (Decimal) | float: round each | float: round final total |
|---|---|---|---|---|
| 300 | 188428.36 | 188428.34 | 188428.36 | 188428.34 |
| 6,000 | 3559846.68 | 3559845.75 | 3559846.68 | 3559845.75 |
| 150,000 | 89221010.74 | 89220991.16 | 89221010.74 | 89220991.16 |

A farmer can only be paid a whole number of paise, so the bank really transfers the sum of the **individually rounded** payments. Rounding only the final total gives a different number (about Rs 19.58 less for 150,000 payments in this experiment), which would not match what was actually paid to the farmers. **Therefore round every payment to 2 decimals (preferably with Decimal) and add the rounded values.** (The test payments here are synthetic, with 4 decimal places before rounding; the point - rounding position changes the total - holds for real data.)

**Additional question - rate table (dictionary) vs recalculation**

**How it works:** compute the rate once for every possible (fat, snf) pair and store it in a dictionary keyed by the pair; then each entry just looks the rate up: rate = table[(fat, snf)]. With 0.1% resolution there are 51 fat values (2.0-7.0) x 31 SNF values (7.0-10.0) = **k = 1,581** pairs. **Time per entry:** recalculation about 493 ns; dictionary lookup about 91 ns (O(1) average) - roughly 5 times faster. Building the table costs about 1.0 ms once (O(k)); it pays off after roughly 2,500 entries (1.0 ms divided by the ~0.4 microsecond saved per entry). **Extra space:** O(k) - the dictionary structure alone was 73,808 bytes for k = 1,581 (plus the key and value objects). **Limitation:** it only works when fat/SNF come from a small finite set; with the 2-decimal analyser readings of Task 12 there would be about 501 x 301 = 150,801 pairs, so the table becomes large and recalculation is better.

# TASK 11 - Recommend and Defend the Final Design

```
"""Recommended final design: Decimal arithmetic, rate card as data, ROUND_HALF_UP to paise."""
from decimal import Decimal, ROUND_HALF_UP, ROUND_HALF_EVEN

PAISE = Decimal("0.01")
RATE_CARD = {
    "base": Decimal("30.00"), "fat_step": Decimal("0.40"), "snf_step": Decimal("0.30"),
    "std_fat": Decimal("3.5"), "std_snf": Decimal("8.5"), "step": Decimal("0.1"),
}
LIMITS = {"fat": (Decimal("2.0"), Decimal("7.0")), "snf": (Decimal("7.0"), Decimal("10.0"))}

def money(x):
    """Round to whole paise, halves go up (conventional commercial rounding)."""
    return x.quantize(PAISE, rounding=ROUND_HALF_UP)

def steps(value, standard, step):
    """Whole number of 0.1% steps (nearest step, halves up)."""
    return int(((value - standard) / step).quantize(Decimal("1"), rounding=ROUND_HALF_UP))

def rate_per_litre(fat, snf, card=RATE_CARD):
    fs = steps(fat, card["std_fat"], card["step"])
    ss = steps(snf, card["std_snf"], card["step"])
    return money(card["base"] + fs * card["fat_step"] + ss * card["snf_step"])

def payout(litres, fat, snf, card=RATE_CARD):
    return money(litres * rate_per_litre(fat, snf, card))

def validate(litres, fat, snf):
    if litres < 0: raise ValueError("litres negative")
    if not (LIMITS["fat"][0] <= fat <= LIMITS["fat"][1]): raise ValueError("fat out of range")
    if not (LIMITS["snf"][0] <= snf <= LIMITS["snf"][1]): raise ValueError("snf out of range")
```

```
"""Task 11 - evidence for the final design: correctness, scalability, JSON rate-card loading."""
import json, os, time, timeit, random
from decimal import Decimal
import milkrate_decimal as md

print("=== correctness against the assignment's expected output ===")
for l, f, s in [("10", "3.5", "8.5"), ("8.5", "4.2", "8.7"), ("12", "3.1", "8.3")]:
    L, F, S = Decimal(l), Decimal(f), Decimal(s)
    print(f"{l:>5} L fat {f} snf {s}  rate {md.rate_per_litre(F, S):6}  pay {md.payout(L, F, S):8}")

print("\n=== scalability: time to price n entries with the Decimal design ===")
random.seed(7)
for n in (300, 6000, 150000):
    data = [(Decimal(random.randint(50, 300)) / 10, Decimal(random.randint(20, 70)) / 10,
             Decimal(random.randint(70, 100)) / 10) for _ in range(n)]
    t0 = time.perf_counter()
    total = sum(md.payout(l, f, s) for l, f, s in data)
    dt = time.perf_counter() - t0
    print(f"n={n:>7}: {dt*1000:8.1f} ms  ({dt/n*1e6:5.2f} microseconds/entry)  total Rs {total}")
print("district-union, 10 days x 2 shifts x 150,000 farmers = 3,000,000 entries per payment cycle")

print("\n=== Additional question: rate card in a JSON file loaded at start-up ===")
card = {"base": "30.00", "fat_step": "0.40", "snf_step": "0.30"}
with open("ratecard.json", "w") as fh:
    json.dump(card, fh)
loaded = json.load(open("ratecard.json"))
print("loaded:", loaded)
for t in (3, 1000, 100000):
    blob = json.dumps({f"k{i}": "1.25" for i in range(t)})
    dt = timeit.timeit(lambda: json.loads(blob), number=20) / 20
    print(f"t={t:>7} rate values: file text {len(blob):>8} bytes, load time {dt*1e6:10.1f} microseconds")
os.remove("ratecard.json")
```

Output (task11_evaluation.py):

```
=== correctness against the assignment's expected output ===
   10 L fat 3.5 snf 8.5  rate  30.00  pay   300.00
  8.5 L fat 4.2 snf 8.7  rate  33.40  pay   283.90
   12 L fat 3.1 snf 8.3  rate  27.80  pay   333.60

=== scalability: time to price n entries with the Decimal design ===
n=    300:      0.8 ms  ( 2.61 microseconds/entry)  total Rs 175872.60
n=   6000:     16.1 ms  ( 2.68 microseconds/entry)  total Rs 3581138.47
n= 150000:    416.4 ms  ( 2.78 microseconds/entry)  total Rs 89184590.35
district-union, 10 days x 2 shifts x 150,000 farmers = 3,000,000 entries per payment cycle

=== Additional question: rate card in a JSON file loaded at start-up ===
loaded: {'base': '30.00', 'fat_step': '0.40', 'snf_step': '0.30'}
t=      3 rate values: file text       42 bytes, load time        1.7 microseconds
t=   1000 rate values: file text    15890 bytes, load time      120.3 microseconds
t= 100000 rate values: file text  1788890 bytes, load time    37778.6 microseconds
```

**Recommended design**

**1. Data types and rounding:** use **Decimal created from strings** for fat %, SNF %, rates and money (or integer paise), integers for step counts and days, str for IDs and shift. Round every payment to paise with ROUND_HALF_UP (the usual commercial rule), then add the rounded payments (Task 10). Keep the rate card as named data (module constants / dictionary) separate from the formulas.

**2. Safe rate revision:** change the values in the single rate-card place only (never inside formulas or functions); record the effective date and keep the old card for re-processing old periods; rerun the automated tests (Task 9/12) before using the new rates; do not change rates in the middle of a payment cycle.

**3. Limitations:** the SNF range (7.0-10.0) is my assumption; the step rule (nearest whole step) loses information if readings have more decimals (see Task 12); Decimal is slower than float (about 2.5 microseconds per entry measured); all data is processed in memory with no database or audit trail; the ten-day period is assumed not to cross a rate-change date.

**4. Can it support 150,000 farmers?** Yes for computation. Measured: 150,000 entries are priced in about 0.39 s (2.6 microseconds per entry, growing linearly: 300 -> 0.8 ms, 6,000 -> 14.6 ms). One ten-day cycle for 150,000 farmers is about 150,000 x 20 = 3,000,000 entries, estimated by linear extrapolation at about 8 seconds, which is acceptable for a batch job. The weak point is memory if all 3,000,000 entries are loaded as Decimal tuples at once (rough estimate: more than 1 GB), so the data should be read and processed in a stream (file/database row by row, running total per farmer), as in Task 8.

**5. Justification from experiments:** Task 2 and 10 showed float totals drift (e.g. 33400.30000000061) while Decimal is exact; Task 10 showed that rounding each payment matches the real bank transfer; Task 9 tests pass; the scalability test above shows linear time; Task 7's module design means a rate change is made in one place.

**Additional question - rate card stored in a text/JSON file loaded at start-up**

**How it works:** the rates are kept in a file such as {'base': '30.00', 'fat_step': '0.40', 'snf_step': '0.30'}; at start-up the program reads the file with json.load() into a dictionary and converts the strings to Decimal. **Start-up time:** O(t) for t rate values - measured 1.8 microseconds for 3 values, 0.2 ms for 1,000 and about 30 ms for 100,000. **Space:** O(t) (the 100,000-value file was about 1.8 MB). **Maintenance:** rates can be revised by editing the file with no change to Python code and no redistribution of the program after each rate revision; the cost is that the file can be missing or malformed, so the program must validate it when loading (store numbers as strings or use parse_float=Decimal so no float errors are introduced). For the few rate values here the load cost is negligible, so file-based rates are a good improvement for maintainability.

## TASK 12 - DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

**A. Impact Analysis**

**1. What changed:** (i) the analyser gives fat and SNF with **two decimal places** (e.g. 3.77%), so readings are finer than the 0.1% step; (ii) from April to June the base rate is Rs 32.00 and each fat step is worth Rs 0.45 (SNF step unchanged at 0.30); (iii) a farmer supplying **more than 500 litres in ten days** receives a bonus of Rs 0.50 per litre.

**2. What must be modified:** *Constants:* one rate card is no longer enough - two cards (normal and April-June) plus BONUS_THRESHOLD_LITRES = 500 and BONUS_PER_LITRE = 0.50. *Expressions:* step counts can no longer be rounded to a whole step (3.77% is 2.7 steps). *Functions:* rate_per_litre and payout need the rate card as a parameter; a new select_rate_card(month) and bonus(total_litres) are needed; ten_day_total must add the bonus; validation must accept 2-decimal readings.

**3. Effect of two-decimal readings on steps and rounding:** step counts become fractional (0.27 / 0.1 = 2.7). If the old nearest-whole-step rule is kept, nearby readings get the same price and prices are off by up to half a step (Rs 0.20/litre for fat), and exact ties (x.5 steps) arise. Float arithmetic also makes these fractional steps inexact, so Decimal is needed. Rates such as 33.605 need an explicit rounding rule (half up, to paise).

**B. Evaluation of the original design (worked examples with the new readings, normal rate card)**

| Fat | SNF | Old whole-step rate | Proportional rate | Difference / litre |
|---|---|---|---|---|
| 3.77 | 8.63 | 31.50 | 31.47 | -0.03 |
| 3.54 | 8.46 | 30.00 | 30.04 | +0.04 |
| 4.25 | 8.55 | 33.50 | 33.15 | -0.35 |
| 3.50 | 8.50 | 30.00 | 30.00 | 0.00 |

**Still appropriate:** named constants (concept), the separate module, the small functions (rate_per_litre, payout, ten_day_total), range validation, rounding each payment, the automated tests. **No longer appropriate:** a single set of rate constants, whole-step rounding, float arithmetic, and a ten_day_total without bonus/season. The design structure is sound, so a modification (not a complete redesign) is justified.

**C. Adaptation - revised design (milkrate_v2.py)**

```
"""Task 12 - revised design: seasonal rate cards, two-decimal analyser readings, ten-day bonus.
All money/percentage values are Decimal (created from strings) to avoid float errors."""
from decimal import Decimal, ROUND_HALF_UP
```

```
PAISE = Decimal("0.01")
STEP = Decimal("0.1")
STANDARD_FAT, STANDARD_SNF = Decimal("3.5"), Decimal("8.5")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("7.0"), Decimal("10.0")          # assumed

NORMAL_CARD = {"base": Decimal("30.00"), "fat_step": Decimal("0.40"), "snf_step": Decimal("0.30")}
SUMMER_CARD = {"base": Decimal("32.00"), "fat_step": Decimal("0.45"), "snf_step": Decimal("0.30")}
SUMMER_MONTHS = (4, 5, 6)                                    # April to June

BONUS_THRESHOLD_LITRES = Decimal("500")                      # strictly more than 500 L in ten days
BONUS_PER_LITRE = Decimal("0.50")

def money(x):
    return x.quantize(PAISE, rounding=ROUND_HALF_UP)

def select_rate_card(month):
    """Return the rate card that applies in the given month number (1-12)."""
    return SUMMER_CARD if month in SUMMER_MONTHS else NORMAL_CARD

def validate_reading(fat, snf):
    if not (MIN_FAT <= fat <= MAX_FAT):
        raise ValueError(f"fat {fat} outside {MIN_FAT}-{MAX_FAT}")
    if not (MIN_SNF <= snf <= MAX_SNF):
        raise ValueError(f"snf {snf} outside {MIN_SNF}-{MAX_SNF}")

def rate_per_litre(fat, snf, card, whole_steps=False):
    """Rate in Rs/litre. whole_steps=True reproduces the old nearest-whole-step rule;
    default (False) applies the step value proportionally, so 3.77% fat = 2.7 steps."""
    fat_steps = (fat - STANDARD_FAT) / STEP
    snf_steps = (snf - STANDARD_SNF) / STEP
    if whole_steps:
        fat_steps = fat_steps.quantize(Decimal("1"), rounding=ROUND_HALF_UP)
        snf_steps = snf_steps.quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    return money(card["base"] + fat_steps * card["fat_step"] + snf_steps * card["snf_step"])

def payout(litres, fat, snf, card, whole_steps=False):
    """Payment for one entry, rounded to paise."""
    validate_reading(fat, snf)
    if litres < 0:
        raise ValueError("litres negative")
    return money(litres * rate_per_litre(fat, snf, card, whole_steps))

def bonus(total_litres):
    """Bonus for ten-day supply: Rs 0.50 on every litre if total litres > 500, else 0."""
    if total_litres > BONUS_THRESHOLD_LITRES:
        return money(total_litres * BONUS_PER_LITRE)
    return Decimal("0.00")

def ten_day_total(entries, month):
    """entries: list of (litres, fat, snf) as Decimal. Returns (payments, bonus, total)."""
    card = select_rate_card(month)
    payments = sum((payout(l, f, s, card) for l, f, s in entries), Decimal("0.00"))
    total_litres = sum((l for l, _, _ in entries), Decimal("0"))
    b = bonus(total_litres)
    return payments, b, payments + b
```

### Test program and results (task12_tests.py)

```
"""Task 12 - tests of the revised design (original rates, seasonal rates, with/without bonus)."""
from decimal import Decimal as D
import milkrate_v2 as m

print("=== Impact of 2-decimal readings: whole-step rule vs proportional rule (normal card) ===")
print(f"{'fat':>5} {'snf':>5} | {'old whole-step rate':>19} | {'proportional rate':>17} | {'difference/L':>12}")
for f, s in [("3.77", "8.63"), ("3.54", "8.46"), ("4.25", "8.55"), ("3.50", "8.50")]:
    a = m.rate_per_litre(D(f), D(s), m.NORMAL_CARD, whole_steps=True)
    b = m.rate_per_litre(D(f), D(s), m.NORMAL_CARD)
    print(f"{f:>5} {s:>5} | {str(a):>19} | {str(b):>17} | {str(b - a):>12}")

print("\n=== Float problem with 2-decimal readings (why Decimal is kept) ===")
print("(4.2 - 3.5) / 0.1 in float =", (4.2 - 3.5) / 0.1, "| in Decimal =", (D("4.2") - D("3.5")) / D("0.1"))

print("\n=== Test table (expected values calculated by hand) ===")
cases = [
    ("T1 original rates (Oct), 3.5/8.5",       lambda: m.payout(D("10"), D("3.5"),  D("8.5"),  m.select_rate_card(10)), D("300.00")),
    ("T2 summer base rate (May), 3.5/8.5",     lambda: m.payout(D("10"), D("3.5"),  D("8.5"),  m.select_rate_card(5)),  D("320.00")),
    ("T3 summer fat step 0.45, fat 4.0",       lambda: m.payout(D("10"), D("4.0"),  D("8.5"),  m.select_rate_card(5)),  D("342.50")),
    ("T4 two-decimal 3.77/8.63 (May)",         lambda: m.payout(D("10"), D("3.77"), D("8.63"), m.select_rate_card(5)),  D("336.10")),
    ("T5 April boundary month, 3.5/8.5",       lambda: m.payout(D("10"), D("3.5"),  D("8.5"),  m.select_rate_card(4)),  D("320.00")),
    ("T6 July (after season), 3.5/8.5",        lambda: m.payout(D("10"), D("3.5"),  D("8.5"),  m.select_rate_card(7)),  D("300.00")),
    ("T7 exactly 500 L -> no bonus (May)",     lambda: m.ten_day_total([(D("50"), D("3.5"), D("8.5"))] * 10, 5)[2],     D("16000.00")),
    ("T8 550 L -> bonus 275.00 (May)",         lambda: m.ten_day_total([(D("55"), D("3.5"), D("8.5"))] * 10, 5)[2],     D("17875.00")),
    ("T9 550 L, bonus with original rates",    lambda: m.ten_day_total([(D("55"), D("3.5"), D("8.5"))] * 10, 10)[2],    D("16775.00")),
    ("T10 400 L, no bonus (Oct)",              lambda: m.ten_day_total([(D("40"), D("3.5"), D("8.5"))] * 10, 10)[2],    D("12000.00")),
]
ok_all = True
print(f"{'Test':42} | {'Expected':>10} | {'Actual':>10} | Result")
for name, fn, exp in cases:
    act = fn()
    ok = act == exp
    ok_all &= ok
    assert ok, name
    print(f"{name:42} | {str(exp):>10} | {str(act):>10} | {'PASS' if ok else 'FAIL'}")
try:
```

```
    m.payout(D("10"), D("7.5"), D("8.5"), m.NORMAL_CARD)
except ValueError as e:
    print(f"{'T11 fat 7.5 rejected':42} | {'ValueError':>10} | {'ValueError':>10} | PASS  ({e})")
print("\nAll tests passed:", ok_all)
```

```
=== Impact of 2-decimal readings: whole-step rule vs proportional rule (normal card) ===
  fat   snf | old whole-step rate | proportional rate | difference/L
 3.77  8.63 |               31.50 |             31.47 |        -0.03
 3.54  8.46 |               30.00 |             30.04 |         0.04
 4.25  8.55 |               33.50 |             33.15 |        -0.35
 3.50  8.50 |               30.00 |             30.00 |         0.00

=== Float problem with 2-decimal readings (why Decimal is kept) ===
(4.2 - 3.5) / 0.1 in float = 7.000000000000002 | in Decimal = 7

=== Test table (expected values calculated by hand) ===
Test                                      |   Expected |     Actual | Result
T1 original rates (Oct), 3.5/8.5          |     300.00 |     300.00 | PASS
T2 summer base rate (May), 3.5/8.5        |     320.00 |     320.00 | PASS
T3 summer fat step 0.45, fat 4.0          |     342.50 |     342.50 | PASS
T4 two-decimal 3.77/8.63 (May)            |     336.10 |     336.10 | PASS
T5 April boundary month, 3.5/8.5          |     320.00 |     320.00 | PASS
T6 July (after season), 3.5/8.5           |     300.00 |     300.00 | PASS
T7 exactly 500 L -> no bonus (May)        |   16000.00 |   16000.00 | PASS
T8 550 L -> bonus 275.00 (May)            |   17875.00 |   17875.00 | PASS
T9 550 L, bonus with original rates       |   16775.00 |   16775.00 | PASS
T10 400 L, no bonus (Oct)                 |   12000.00 |   12000.00 | PASS
T11 fat 7.5 rejected                      | ValueError | ValueError | PASS  (fat 7.5 outside 2.0-7.0)

All tests passed: True
```

### D. Evidence-Based Defence

**Decision: modify** the original design (keep module + functions + named constants + validation + tests; change the number type to Decimal, replace the single rate constants by rate cards chosen by month, drop whole-step rounding, add a bonus function). Tests T1-T11 above include the original rates (T1, T6, T9, T10), the seasonal rates (T2-T5), April as the first seasonal month (T5), July as the first normal month (T6), and with/without bonus (T7-T10, including exactly 500 L giving no bonus because the rule says *more than* 500). All 11 passed, so the revised design meets the new requirements.

**Trade-offs:** Decimal is slower than float (about 2.5 microseconds per entry, Task 11) but exact; rate cards add a parameter to every function but make seasonal changes a data change; proportional steps change prices slightly compared with the old rule (table in B) but follow the stated rate (Rs 0.40 per 0.1% means Rs 0.04 per 0.01%).
**Assumptions:** the bonus applies to **all** litres of the ten days (not only the litres above 500) when the total is more than 500; the season is decided by one month number for the whole ten-day period (a period that crosses 31 March/30 June is not handled); the rate is rounded to paise before multiplying by litres (e.g. 3.77/8.63 in May gives 33.605 -> 33.61, payment 336.10); the SNF range 7.0-10.0 is assumed. If the co-operative decides differently on any of these, only the named constants/one function need to change.